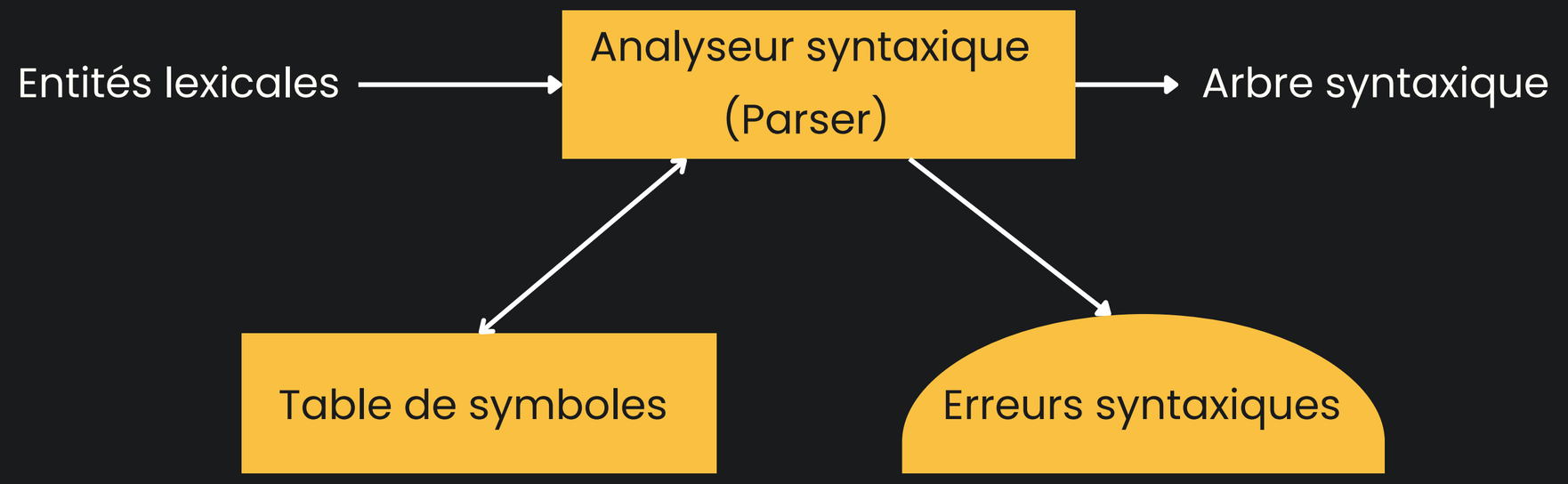
# ANALYSE SYNTAXIQUE

# ANALYSE SYNTAXIQUE

- L'analyse **syntaxique (parsing)** vérifie que les entités lexicales (issues de l'analyse lexicale) sont organisées dans l'ordre correct selon les règles grammaticales du langage.
- Rôle :
  - Vérifier la structure des instructions.
  - Construire un arbre syntaxique.
  - Détecter les erreurs de syntaxe.



# ANALYSE SYNTAXIQUE

Exemples d'erreurs syntaxiques

code.c

```c
#include <stdio.h>

int main() {
    printf("Hello, World!");
    if (m == 1 printf("m est valide");
    return 0;
}
```

code.c

```c
#include <stdio.h>

int main() {
    int x = 5
    printf("x = %d\n", x);
    return 0;
}
```

# ANALYSE SYNTAXIQUE

code.c

```c
#include <stdio.h>

int main() {
    printf("Hello, World!");
    if (m == 1 printf("m est valide");
    return 0;
}
```

- **Erreur syntaxique:** parenthèse fermante manquante dans la condition du **if**.

code.c

```c
#include <stdio.h>

int main() {
    int x = 5
    printf("x = %d\n", x);
    return 0;
}
```

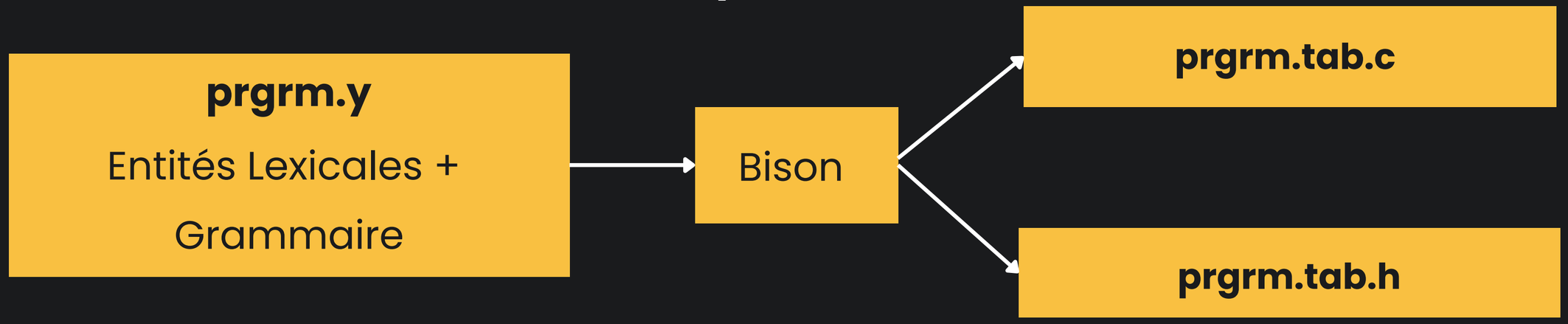
- **Erreur syntaxique:** le point-virgule ; après int x = 5 est obligatoire en C.

# ANALYSE SYNTAXIQUE

- Implémenter un analyseur syntaxique nécessite l'implémentation d'une méthode d'analyse syntaxique vue en cours LL(k), LR(k), SLR(k), LALR(k), etc.
- Ces techniques demandent l'écriture de milliers de lignes de code.
- Heureusement, des outils comme **Bison** automatisent ce processus et génèrent pour nous un analyseur syntaxique à partir de la grammaire du langage.

# BISON

- Bison est un **générateur** de code d'analyseurs syntaxiques LALR (Look-Ahead Left-to-Right).
- Il prend en entrée les entités lexicales produites par l'analyse lexicale et la grammaire du langage à analyser.
- Les fichiers Bison utilisent l'extension « **.y** ».



# FORMAT D'UN FICHIER BISON

Le format d'un fichier Bison est similaire à celui de Flex. Il est composé de trois parties séparées par '**%%**'

```
%{
Définitions en langage C
%}
Les définitions des terminaux et d'axiome
%%
Les règles de la grammaire
%%
Redéfinitions des fonctions prédéfinies
```

Partie 1

Partie 2

Partie 3

# Partie 1: Définitions

```
%{
Définitions en langage C
%}

Les définitions des terminaux et d'axiome
```

**1.1. Définitions en C:** Cette partie contient les en-têtes (Fichiers .h), les déclarations de variables globales, les prototypes de fonctions, ainsi que toute autre déclaration utile au bon fonctionnement de l'analyseur.

**Exemple:**

```
%{
#include <stdio.h>
#include <stdlib.h>
void yyerror(const char *s);
%}
```

# Partie 1: Définitions

**1.2. Définitions des terminaux et des axiomes:** Cette partie contient toutes les déclarations nécessaires pour la grammaire à savoir:

**a)** Déclaration des symboles terminaux

**b)** Définition des priorités et d'associativité

**c)** Autres déclarations

## 1.2.a) Déclaration des symboles terminaux

**Rappel:** Les symboles terminaux sont les mots que l'analyseur lexical (Flex) reconnaît et transmet à Bison.

- Ceci est effectué en utilisant le mot clé **%token**.

**Exemple:**

- **%token** MC_if idf pvg cst

On peut aussi spécifier le type d'un terminal par:

%token<type> terminal

**Exemple:**

- %token**<int>** entier
- %token**<chaine>** idf

## 1.2.b) Définition des priorités et d'associativité

- L'**associativité** : On utilise les mots-clés suivants :

  - %left: Spécifie que l'opérateur est **associatif à gauche.**

**Exemple:** % left '-' => l'expression a - b - c sera interprétée comme (a - b) - c.

  - %right: Spécifie que l'opérateur est **associatif à droite**.

**Exemple:** % right '=' => L'expression a = b = c sera évaluée comme a = (b = c).

  - %nonassoc: Spécifie que l'opérateur est non-associatif. Cela indique que l'opérateur ne peut pas être utilisé de manière répétée sans parenthèses.

**Exemple :** %nonassoc '<' '>' => une expression comme a < b < c provoquera une erreur de syntaxe, car il est interdit d'enchaîner cet opérateur sans clarification par des parenthèses.

- La **priorité** est définie selon l'ordre de déclaration des unités lexicales.

# Exemple Priorité & associativité

**On a la déclaration suivante :**

```
%right '='
%left '+' '-'
%left '*'  '/'
```

↓ **Priorité croissante**

**-> Exemple d'entrée:**

**x = y = p + q * r - s / t**

**-> Analyse**

**x = ( y = ( (p + (q * r)) - (s / t) ) )**

## 1.2.c) Autres déclarations

- **%start :** permet de définir l'axiome (non-terminal initial) de la grammaire.

-> En l'absence de cette déclaration, Bison considère le premier non-terminal de la grammaire en tant que son axiome.

- **%type** : définir un type à un symbole **non-terminal.**

**Exemple:**
**%type <Entier> EXPR**
**// tel que EXPR est un non-terminal.**

- **%union :** permet de spécifier tous les types possibles pour les valeurs sémantiques

```
%union {
int entier;
char* str;
float numvrg;
}
```

# PARTIE 2 : LES RÈGLES DE PRODUCTION

- Les règles de grammaire définissent la structure syntaxique du langage, en précisant comment les différents symboles peuvent être combinés pour former des règles valides du programme.
- Dans cette partie, nous décrivons la grammaire LALR(1) du langage à compiler ainsi que les routines sémantiques à exécuter, en suivant la syntaxe suivante:

**<symbole NonTerminal> : <règle de dérivation 1> {action 1 en langage C }**
**| <règle de dérivation 2> {action 2 en langage C }**
**| .....**
**| <règle de dérivation N> {action N en langage C}**
**;**

**Exemple:** %token a b

```
%%
S:aAb {printf("règle de dérivation ");};
```

# Partie 3: code C

- Cette partie représente le code principal de l'analyseur syntaxique.
- Elle contient la fonction main() ainsi que les définitions des fonctions nécessaires au bon fonctionnement du programme.
- Elle doit inclure les deux fonctions suivantes :

```
main ()
{ yyparse();
}
yywrap(){}
```

# LIEN ENTRE FLEX ET BISON

Afin de lier FLEX à BISON, On doit ajouter dans la partie C du FLEX l'instruction suivante:

```
%{
# include "Pgrm.tab.h "
%}
```

# EXEMPLE: CREATION D'UN COMPILATEUR LEXICAL/SYNTAXIQUE POUR : X=5;

## lexical.l

```
%{
#include "syntax.tab.h"
int nb_ligne=1;
%}
lettre [a-zA-Z]
chiffre [0-9]
IDF {lettre}({lettre}|{chiffre})*
cst {chiffre}+
%%
{IDF} return idf;
{cst} return cst;
= return aff;
; return pvg;
[ \t]
\n {nb_ligne++; }
. printf("erreur lexicale à la ligne %d \n",nb_ligne) ;
%%
```

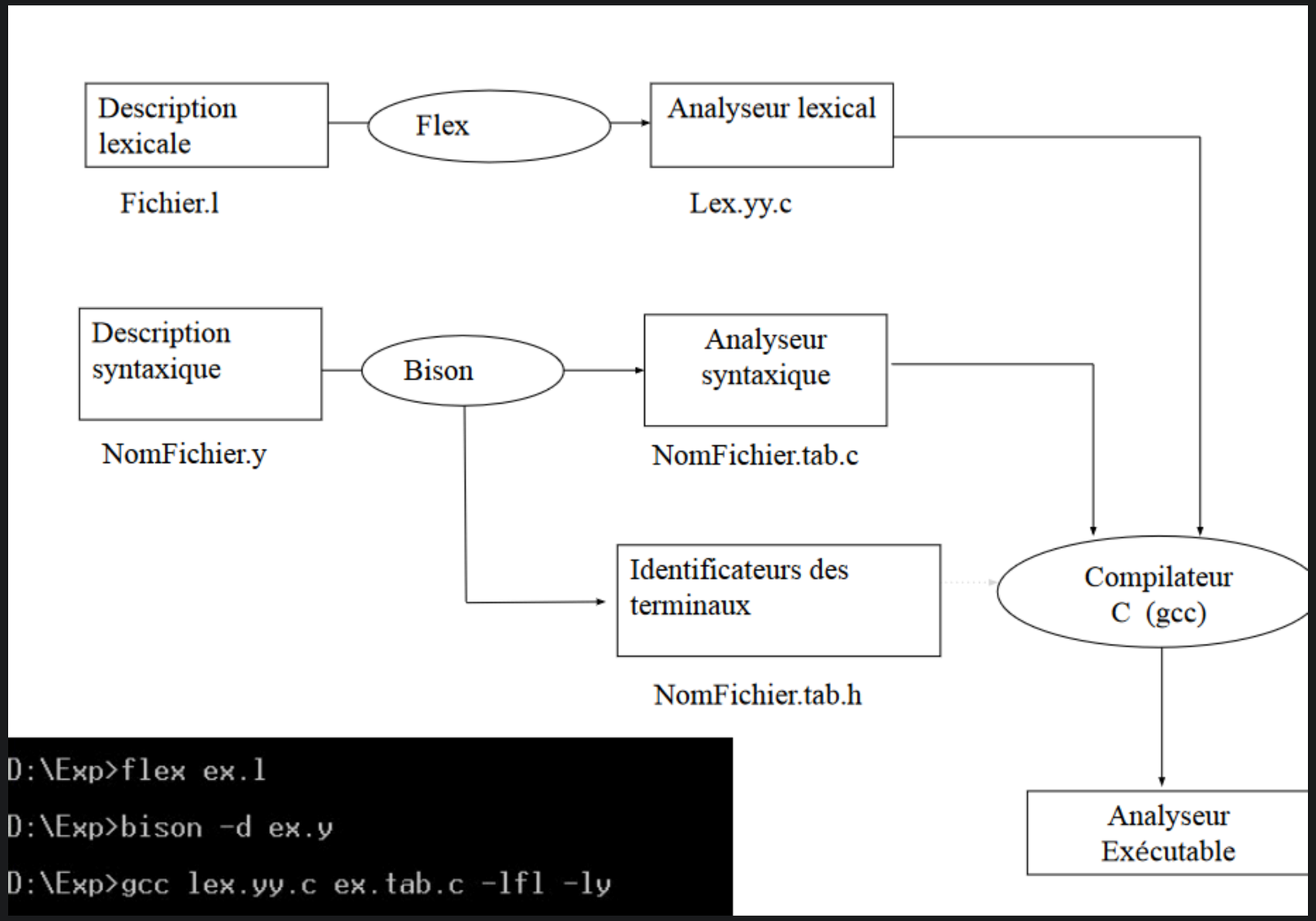
## syntax.y

```
%token cst pvg aff idf
%%
S: idf aff cst pvg {printf("syntaxe correcte");
YYACCEPT;}
;
%%
main ()
{
yyparse();
}
yywrap()
{}
int yyerror(char *msg)
{ printf(" Erreur syntaxique");
return 1;
}
```

# COMMANDES POUR COMPILER ET EXÉCUTER UN PROGRAMME FLEX/BISON

Remarque:

L'option -d est utilisée pour demander à Bison de générer un fichier d'en-tête (header file ".h") contenant les définitions des entités lexicales utilisées dans la grammaire.




```
D:\Exp>flex ex.l
D:\Exp>bison -d ex.y
D:\Exp>gcc lex.yy.c ex.tab.c -lfl -ly
```

# VARIABLES ET FONCTIONS PRÉDÉFINIES DE BISON

- **YYACCEPT :** instruction qui permet de stopper l'analyseur syntaxique en cas de sucées.
- **main () :** elle doit appeler la fonction yyparse (). L'utilisateur doit écrire son propre main dans la partie du bloc principal.
- **yyparse** () : c'est la fonction principale de l'analyseur syntaxique. On doit faire appelle à cette fonction dans la fonction main().
- int **yyerror** ( char* msg ) : lorsque une erreur syntaxique est rencontrée, yyparse fait appelle à cette function. On peut la redéfinir pour donner plus de détails dans le message d'erreur. Par défaut elle est définie comme suit:

```
int yyerror ( char* msg )
{
printf ( " Erreur Syntaxique rencontrée à  la ligne %d: \n ",
nb_ligne );
return 1;
}
```